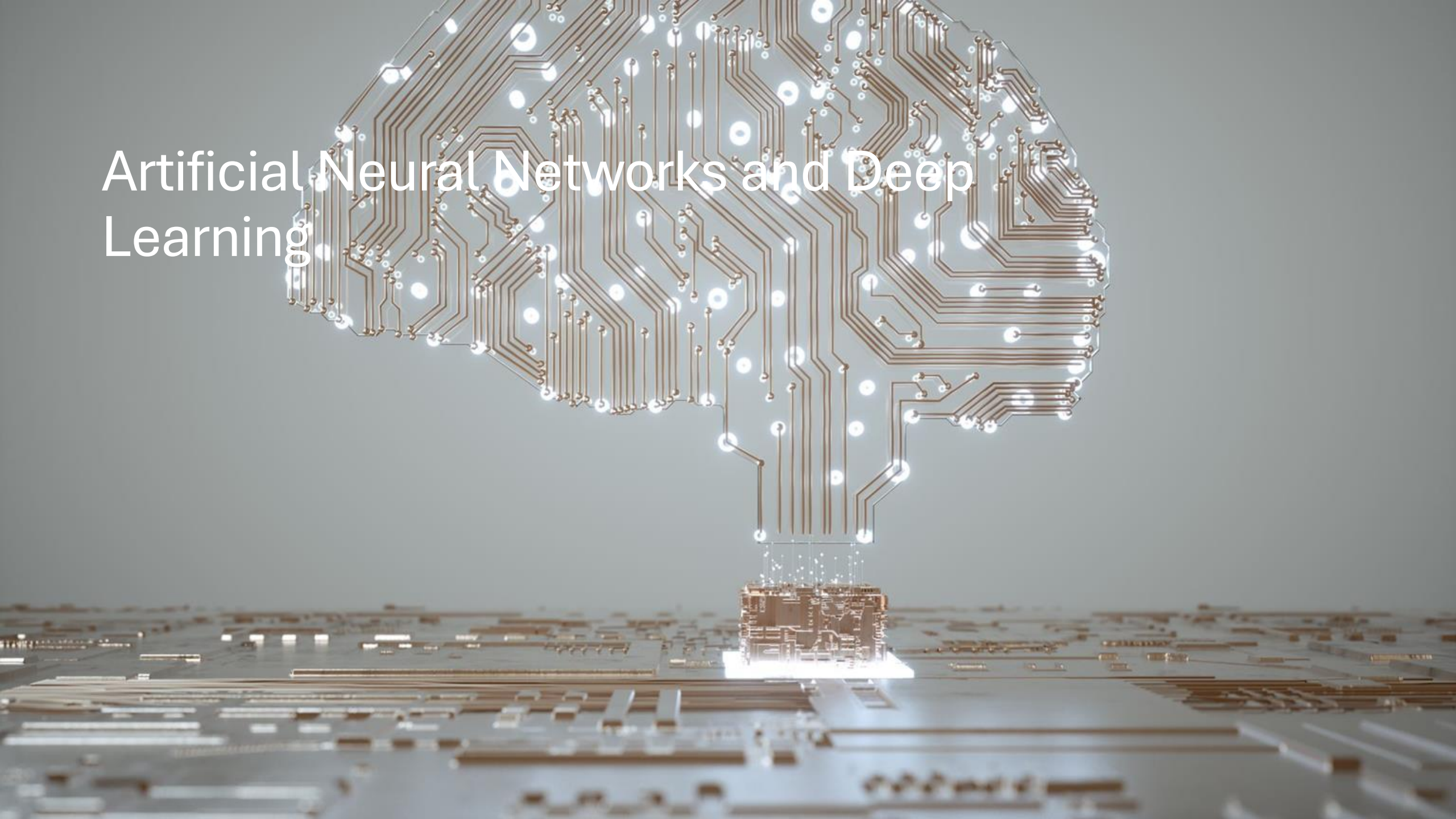
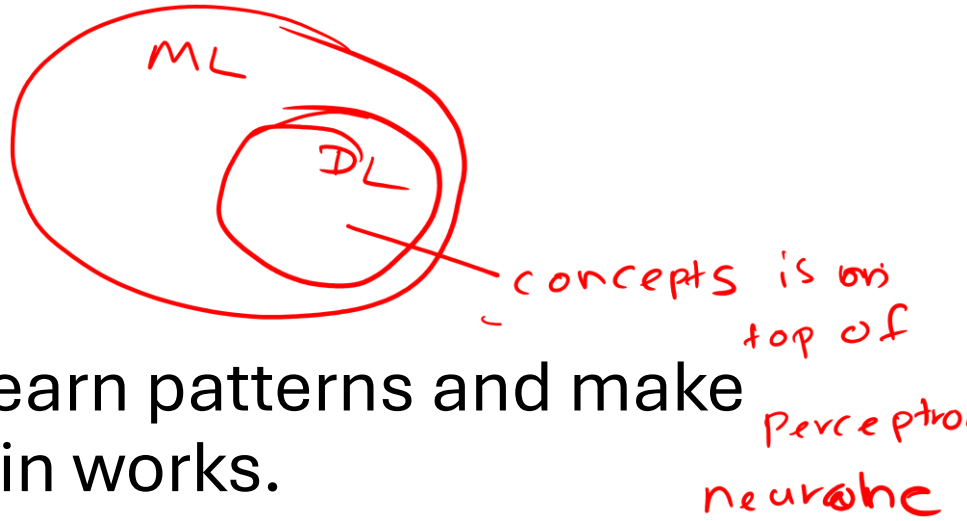
Artificial Neural Networks and Deep Learning



What is Deep Learning?

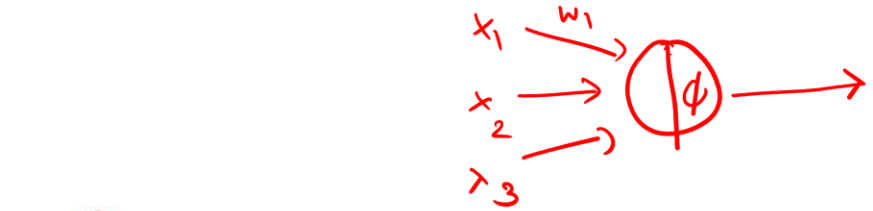


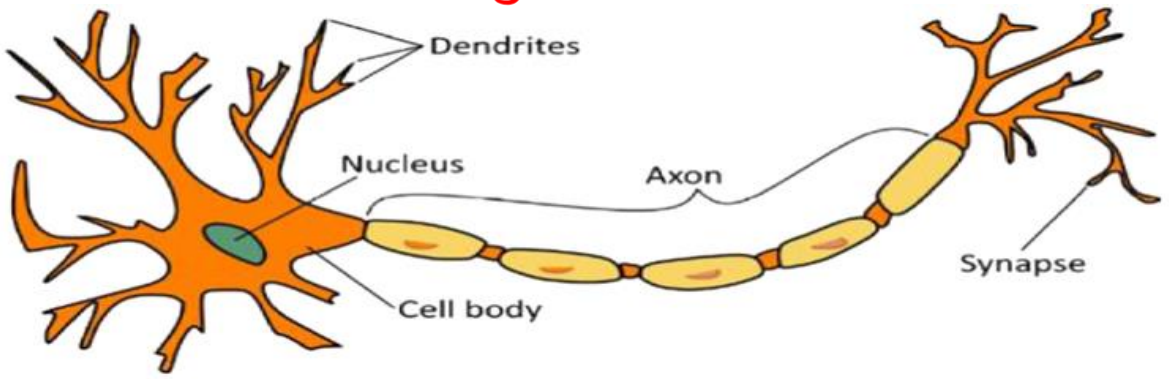
- Deep Learning is a way for computers to learn patterns and make decisions, inspired by how the human brain works.
- **Part of Machine Learning (ML)**:
  - Traditional ML → Algorithms need you to manually select features (e.g., detecting edges in an image).
  - Deep Learning → Learns features automatically (e.g., finds eyes, nose, and face without explicit coding).

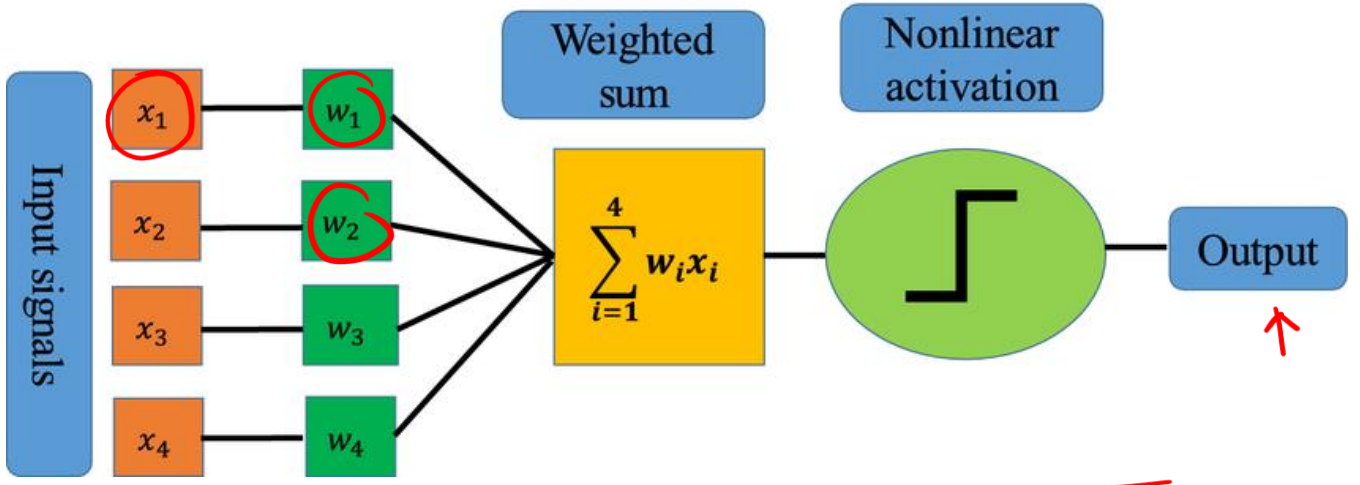
- Think of traditional ML as cooking with a recipe (steps are predefined).
- Deep Learning is like a chef who experiments and learns to cook by tasting, improving over time.

Biological Inspiration

- Neural networks are inspired by the brain's structure:
- Neurons receive signals through dendrites, process them, and transmit signals through axons.







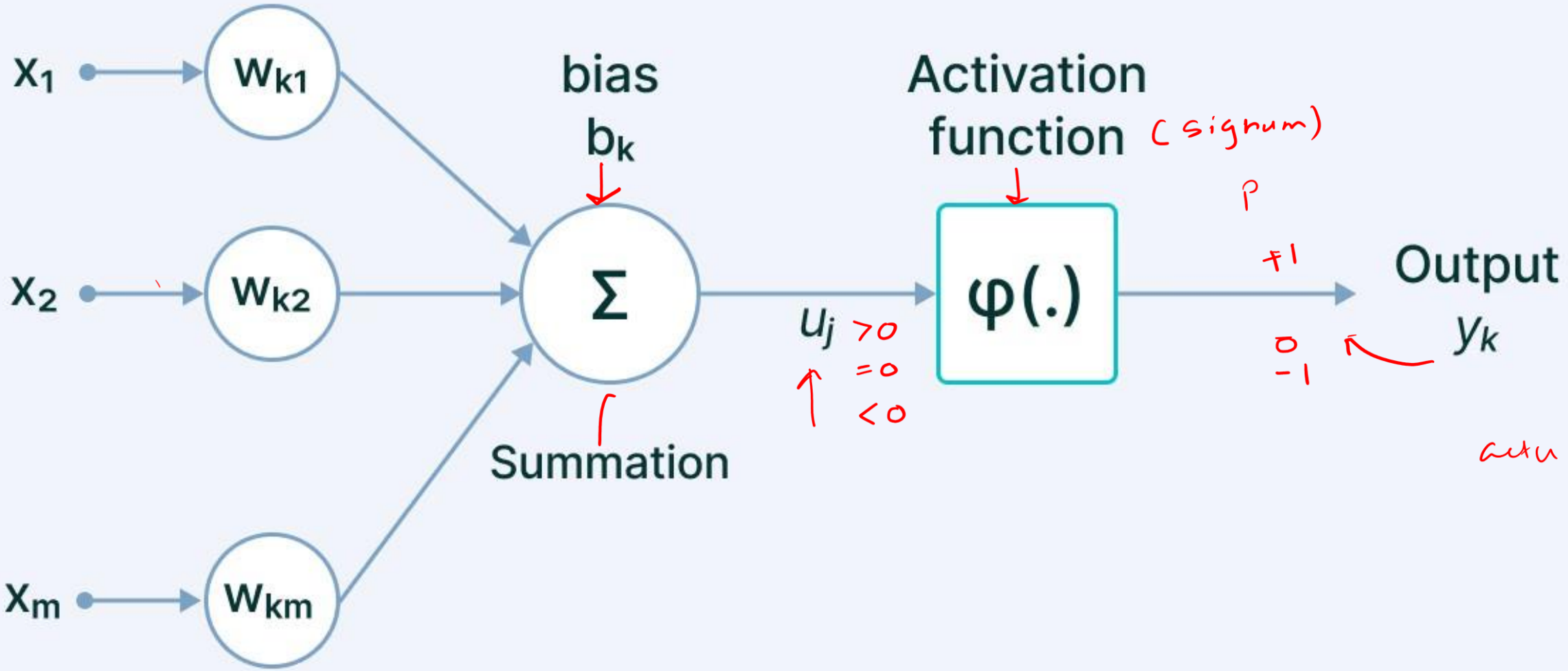
Linear Seperable

Introduction to Neural Networks

What is a Neural Network?

- A neural network is a computational model inspired by the way biological neural networks in the human brain process information.
- It is made up of layers of interconnected nodes or "neurons."

Neuron


x_1
w_{k1}
x_2
w_{k2}
x_m
w_{km}
bias
b_k
Σ
Summation
Activation
function
φ(.)
u_j >0
=0
<0
(signum)
+1
0
-1
Output
y_k
actual

Basic Terminology

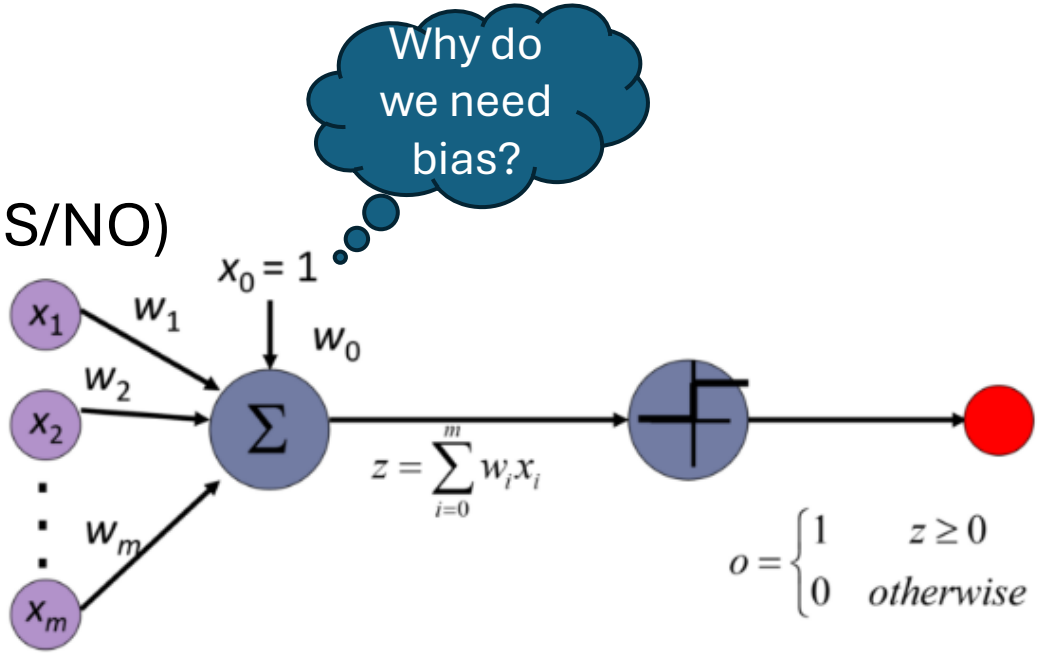
- **Neuron (Node)**: Smallest processing unit.
- **Layer**:
  - Input Layer → raw data enters.
  - Hidden Layer(s) → where learning happens.
  - Output Layer → final prediction.
- **Weights**: Strength of connection between neurons.
- **Bias**: Adjustment term.
- **Activation Function**: Adds non-linearity (so network can solve complex problems).

Perceptron: The First Neural Network

- The perceptron is one of the earliest types of artificial neurons introduced by Frank Rosenblatt in 1958.
- It is **a binary classifier** that determines whether an input belongs to one of two classes.
- A perceptron takes several binary or real-valued inputs, applies weights to them, sums the results, adds a bias, and then passes the result through an activation function (usually a step function).
- **Mathematical Representation:** $y = 1$ if $(w \cdot x + b) \geq 0$, else 0

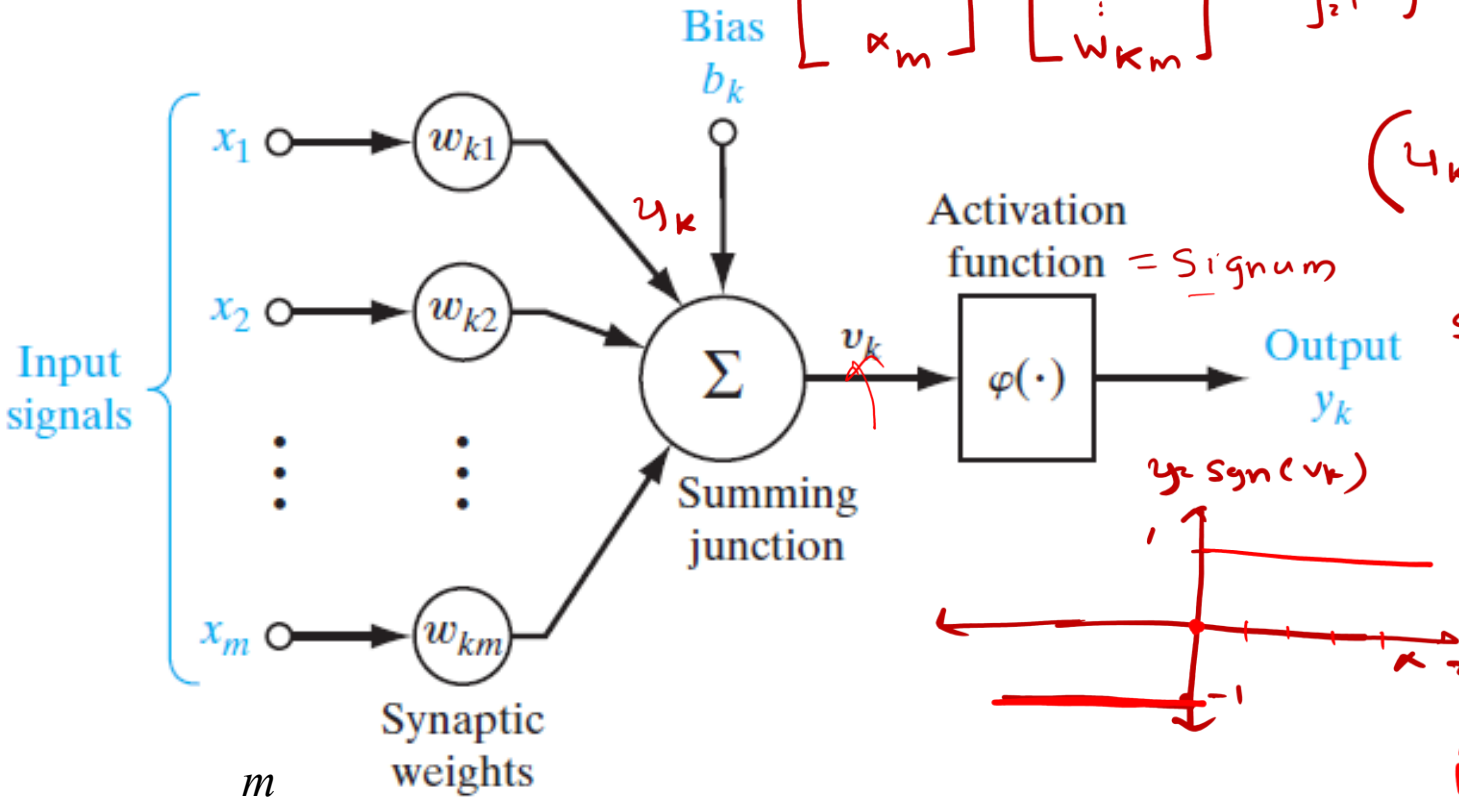
What is a Perceptron?

- A **perceptron** is the simplest form of a neural network.
- It has:
  - **Inputs** (like exam marks, height, etc.)
  - **Weights** (importance of each input)
  - **Bias** (adjusts output up/down)
  - **Activation Function** (decides output: YES/NO)



The perceptron model

Model of a neuron



$$u_k = \sum_{j=1}^{m} w_{kj} x_k$$

$$y_k = \phi(u_k + b_k)$$

$$\begin{bmatrix} x_1 \\ x_2 \\ \\ x_m \end{bmatrix} \cdot \begin{bmatrix} w_{k1} \\ w_{k2} \\ \vdots \\ w_{km} \end{bmatrix} = \sum_{j=1}^{m} w_{kj} x_k = u_k$$

$(u_k + b_k) = v_k \geq 0.67?$

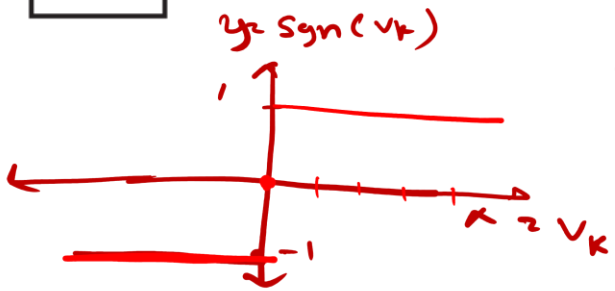
Activation function = Signum

Signum activation = ϕ

if $v_k < 0$ $\quad sgn(v_k) = -1$

$v_k = 0$ $\quad sgn(v_k) = 0$

$v_k > 1$ $\quad sgn(v_k) = 1$

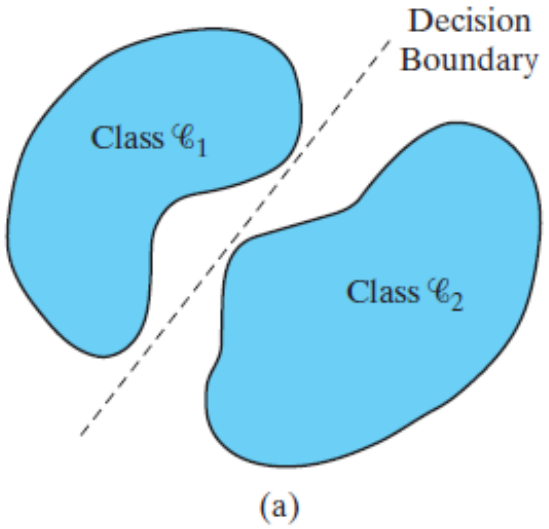


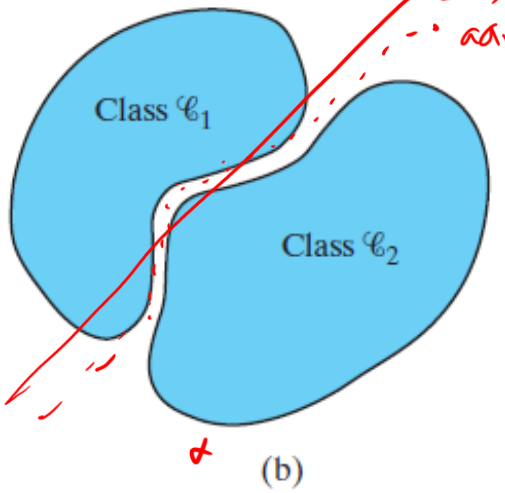
if $v_k = 0.67$

what's $sgn(v_k)$?!

Perceptron

The perceptron is the simplest form of a neural network used for the classification of patterns said to be *linearly separable* (i.e., patterns that lie on opposite sides of a hyperplane).





- The two classes $\mathscr{C}_1$ and $\mathscr{C}_2$ are sufficiently separated from each other for us to draw a hyperplane (in this case, a straight line) as the decision boundary.
- If, however, the two classes $\mathscr{C}_1$ and $\mathscr{C}_2$ are allowed to move too close to each other, as in (b) above, they become nonlinearly separable, a situation that is beyond the computing capability of the perceptron.

- **Mathematical Representation:** $y = 1$ if $(w \cdot x + b) \geq 0$, else 0

Perceptron

- Perceptron consists of a single neuron with adjustable synaptic weights and bias.
- The algorithm used to adjust the free parameters of this neural network first appeared in a learning procedure developed by Rosenblatt (1958, 1962) for his perceptron brain model.
- Rosenblatt proved that if the patterns (vectors) used to train the perceptron are drawn from two linearly separable classes, then the perceptron algorithm converges and positions the decision surface in the form of a hyperplane between the two classes.
- The proof of convergence of the algorithm is known as the ***perceptron convergence theorem*.**

Perceptron Convergence Algorithm

→ linear seperable data.

Variables and Parameters:

$\mathbf{x}(n)$ = $(m+1)$-by-1 input vector
$= [+1, x_1(n), x_2(n), \ldots, x_m(n)]^T$
$\mathbf{w}(n)$ = $(m+1)$-by-1 weight vector
$= [b, w_1(n), w_2(n), \ldots, w_m(n)]^T$
b = bias
$y(n)$ = actual response (quantized)
$d(n)$ = desired response
η = learning-rate parameter, a positive constant less than unity

1. *Initialization.* Set $\mathbf{w}(0) = \mathbf{0}$. Then perform the following computations for time-step $n = 1, 2, \ldots$.
2. *Activation.* At time-step n, activate the perceptron by applying continuous-valued input vector $\mathbf{x}(n)$ and desired response $d(n)$.
3. *Computation of Actual Response.* Compute the actual response of the perceptron as

   actual response / Predicted → $y(n) = \text{sgn}[\mathbf{w}^T(n)\mathbf{x}(n)]$

   where sgn(·) is the signum function.

   Actual result = Predicted or
4. *Adaptation of Weight Vector.* Update the weight vector of the perceptron to obtain

   η [actual result − predicted result] [x]

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$$

   where

   $w_{(n+1)} = w_{(n)} + \eta [\ 0\ ]$

$$d(n) = \begin{cases} +1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_1 \\ -1 & \text{if } \mathbf{x}(n) \text{ belongs to class } \mathscr{C}_2 \end{cases}$$

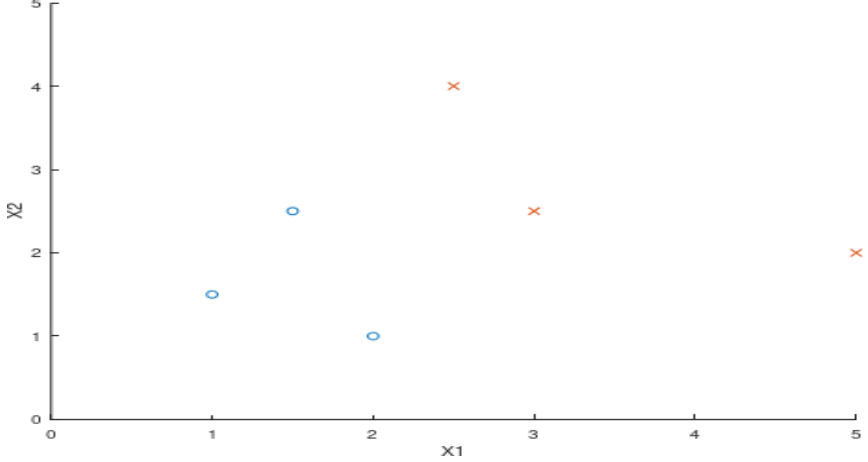
5. *Continuation.* Increment time step n by one and go back to step 2.

Exampl

X1	X2	Class
1	1.5	+1
1.5	2.5	+1
2	1	+1
2.5	4	-1
3	2.5	-1
5	2	-1

Actual

100 000 row



$$X[0] = \begin{bmatrix} +1 \\ +1 \\ +1.5 \end{bmatrix} \qquad W[0] = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$v = w^T X = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}^T \begin{bmatrix} +1 \\ +1 \\ +1.5 \end{bmatrix} = [0 \quad 0 \quad 0] \begin{bmatrix} +1 \\ +1 \\ +1.5 \end{bmatrix} = 0$$

$y_0 = sign(v_0) = sign(W^T x)$

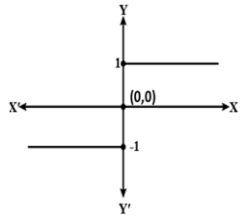
$$y(0) = \text{sign}(0) = 0$$

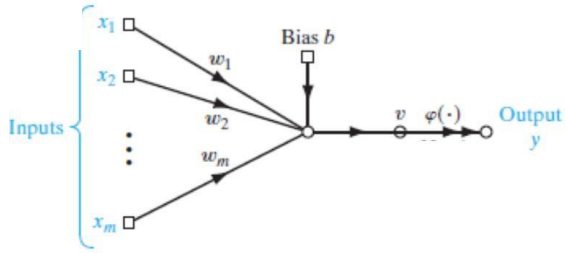
error

$$w(1) = w(0) + \mu[d(0) - y(0)]\, X(0)$$

$$Sgn(x) = \begin{cases} -1, & if\ x < 0 \\ 0, & if\ x = 0 \\ 1, & if\ x > 0 \end{cases}$$

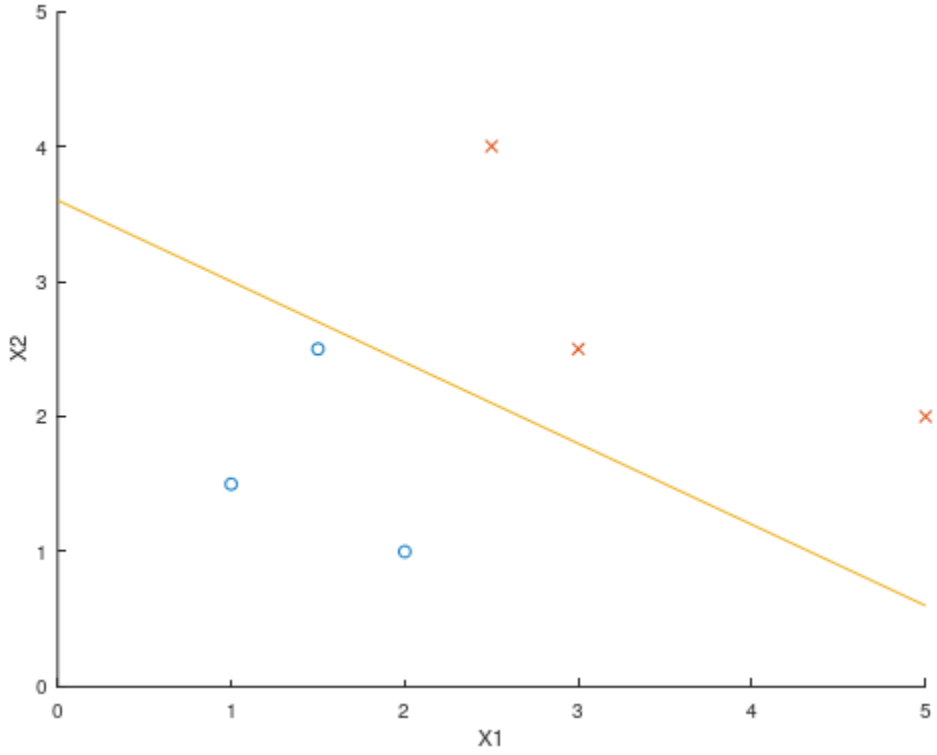
Graph of Signum Function





( )]

b	w1	w2
1.8	-0.3	-0.5



The Batch Perceptron Algorithm

- The previously discussed perceptron convergence algorithm was presented without reference to a cost function. Moreover, the derivation focused on a single-sample correction.
- Introducing the following two will make it generalized.
  - The generalized form of a perceptron cost function
  - use the cost function to formulate a batch version of the perceptron convergence algorithm

When consider about mis classified Samples

$W_{(n)}^T \cdot X_{(n)} > 0$ $\quad C_1 \rightarrow -1$

$W_{(n)}^T \cdot X_{(n)} < 0$ $\quad C_2 \rightarrow 1$

	x_1	x_2	$y_{(n)}$ (Predicted)	$d_{(n)}$ (Actual)	
0	1	1.5	+1	-1	misclassification
1	2	2.5	-1	+1	misclassification

$y_{(n)} = \phi(x_{(n)}^T w_{(n)})$

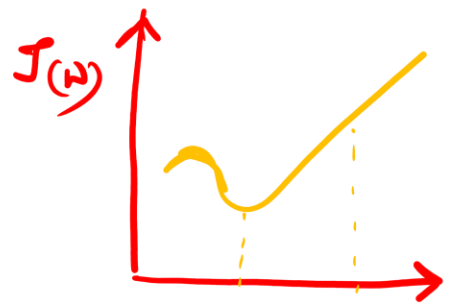
$y_{(n)} \cdot d_{(n)} < 0$

$\text{sgn}(W_{(n)}^T \cdot X_{(n)}) \cdot d_{(n)} < 0$ ← For all misclassified samples

$-W_{(n)}^T \cdot X_{(n)} \cdot d_{(n)} > 0$

$J(W) = \sum_{x \in X} W_{(n)}^T \cdot X_{(n)} \cdot d_{(n)}$ $\quad X \rightarrow$ misclassified Samples

Cost Function



When no of misclassified samples ↑ → J(w) ↑

are we going to select these randomly

Loss optimization

$$W^* = \arg\min_W \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

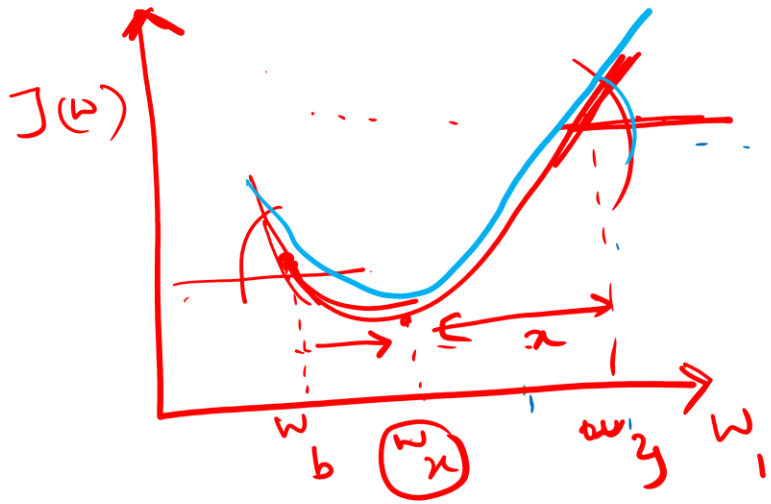
- W: the weights (parameters) of the neural network.
- $f(x^{(i)}; W)$: the model's prediction for input $x^{(i)}$ using weights W.
- $y^{(i)}$: the true label for input $x^{(i)}$.
- $\mathcal{L}(\cdot, \cdot)$: the loss function (e.g., mean squared error, cross-entropy) measuring prediction error.
- $\frac{1}{n} \sum_{i=1}^{n}$ ...: the average loss over all training examples.

$$W^* = \arg\min_W J(W)$$

Here, $J(W)$ is just shorthand for the cost function (the average loss above).

So this equation is a simplified representation of the same idea: *choose the weights W that minimize the cost function.*

Training a neural network is about adjusting weights so the model's predictions are as close as possible to the true outputs, by minimizing the loss (cost) function.



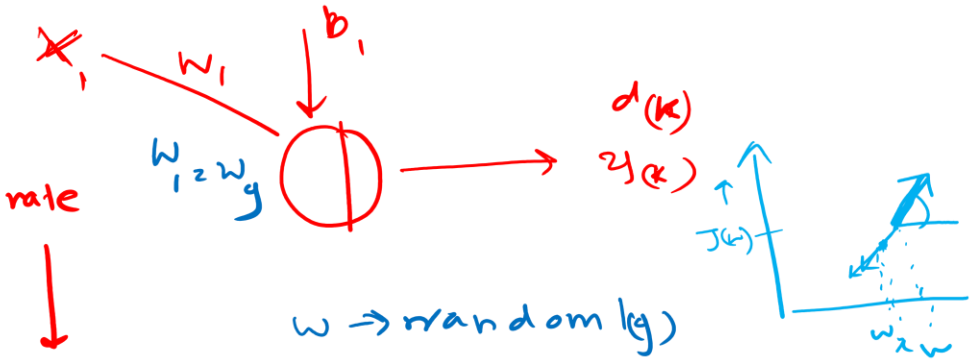
$J(w)\downarrow$ Accuracy $\uparrow$

$$w_x = w_y - \eta g$$

optimum ← w_x ; known ← w_y ; learning rate ← η ; g ↓

started ↑

$$w_x = w_y - g$$



$$w_x = w_b + g \qquad w_x = w - g$$

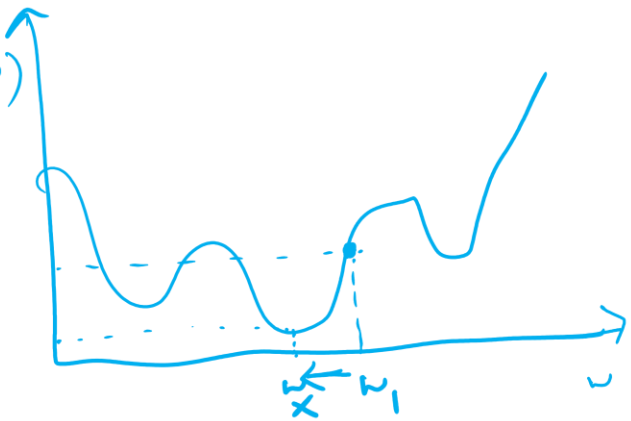
vector

$$w_x = w_b - g$$

$$\frac{\partial J(w)}{\partial (w_y)} = \frac{\partial \left(w^T_{(n)} \cdot x_{(n)} \cdot d_{(n)} \right)}{w} = -x_{(n)} d_{(n)}$$

error

$$w_{(n+1)} = w_{(n)} - \eta \left(x_{(n)} d_{(n)} \right)$$



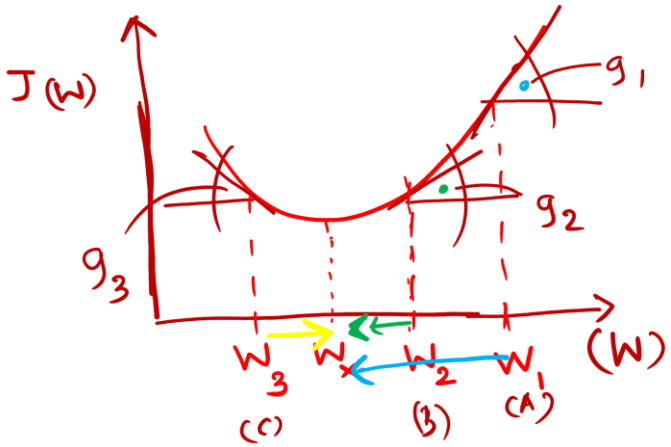
$$w_{(n+1)} = w - g$$

$$= w - \left(\frac{-\partial J(w)}{\partial w} \right)$$

$$= w - \frac{-\partial w_n x_n d_n}{\partial w_n}$$

$$w_{(n+1)} = w + \eta (x_n d_{(n)})$$

$$\boxed{w_{(n+1)} = w_{(n)} + \eta \left(x_{(n)} d_{(n)} \right)}$$



At point A

$$W_x = W_1 - g_1$$

At point B

$$W_x = W_2 - g_2$$

At point C

$$W_x = W_3 + (-g_3)$$

$$W_x = W_3 - \bigcirc$$

Gradient

$$\frac{\partial J(W)}{\partial W} = \frac{-\sum W^T_{(n)} \cdot x_{(n)} \cdot d_{(n)}}{W.}$$

mis

$$= \left(-x_{(n)} d_{(n)}\right)$$

input, desired value

$$W_{(n+1)} = W_{(n)} - \eta\left(-x_{(n)} \cdot d_{(n)}\right)$$

$$W_{(n+1)} = W_{(n)} + \eta\left(x_{(n)} \cdot d_{(n)}\right) \leftarrow 1 \text{ weight/sample}$$

how much you should move in the slope at a time

$$W_{(n+1)} = W_{(n)} - \eta\, -x_n d_n$$

$$W_{(n+1)} = W_{(n)} + \eta\left(x_{(n)} d_{(n)}\right)$$

$$W_{(n+1)} = W_{(n)} + \eta \sum_{x \in \chi} x_{(n)} d_{(n)}$$

— misclassified sam

χ = set of incorrectly classified samples

$$\rightarrow \quad W_{(n)} + \eta \sum_{x \in x_1} x_{(n)} d_{(n)}$$

Batch Perceptron Algorithm

- We can define a cost function for the perceptron as follows:

$$J(\mathbf{w}) = \sum_{\mathbf{x}(n)\in\mathcal{X}} (-\mathbf{w}^T\mathbf{x}(n)d(n)) \quad \text{where } \mathcal{X} \text{ is the set of samples } \mathbf{x} \textit{ misclassified}$$

- If all samples are correctly classified, then the set $\mathcal{X}$ would be empty and $J(w)$ would be zero.
- Differentiate $J(w)$ with respect to w gives the gradient vector

$$\nabla J(\mathbf{w}) = \sum_{\mathbf{x}(n)\in\mathcal{X}} (-\mathbf{x}(n)d(n))$$

Batch Perceptron Algorithm *cont.*

minimize misclassification ≡ minimize loss

- The weight adjustment method that we discussed above is called the *method of steepest descent* as the amount of adjustment depends on the steepness of the cost function (i.e. gradient).

- The weight adjustment happens in the direction opposite to the gradient vector.

- The gradient vector is mathematically denoted b $\nabla J(\mathbf{w})$

$$\nabla J(\mathbf{w}) = \sum_{\mathbf{x}(n) \in \mathcal{X}} (-\mathbf{x}(n) d(n))$$

$\frac{\partial}{\partial w}$

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \eta(n) \nabla J(\mathbf{w})$$
$$= \mathbf{w}(n) + \eta(n) \sum_{\mathbf{x}(n) \in \mathcal{X}} \mathbf{x}(n) d(n)$$

$w_{(n+1)} = w_{(n)} - \eta \left(\frac{\partial}{\partial w} \right)$

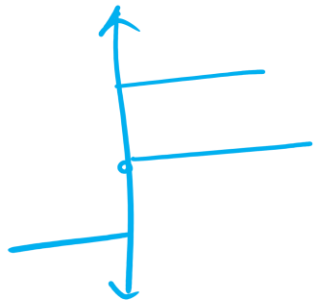
$w_{(n+1)} = w_{(n)} - \eta \sum x_{(n)} d_{(n)}$

misclassified samples.

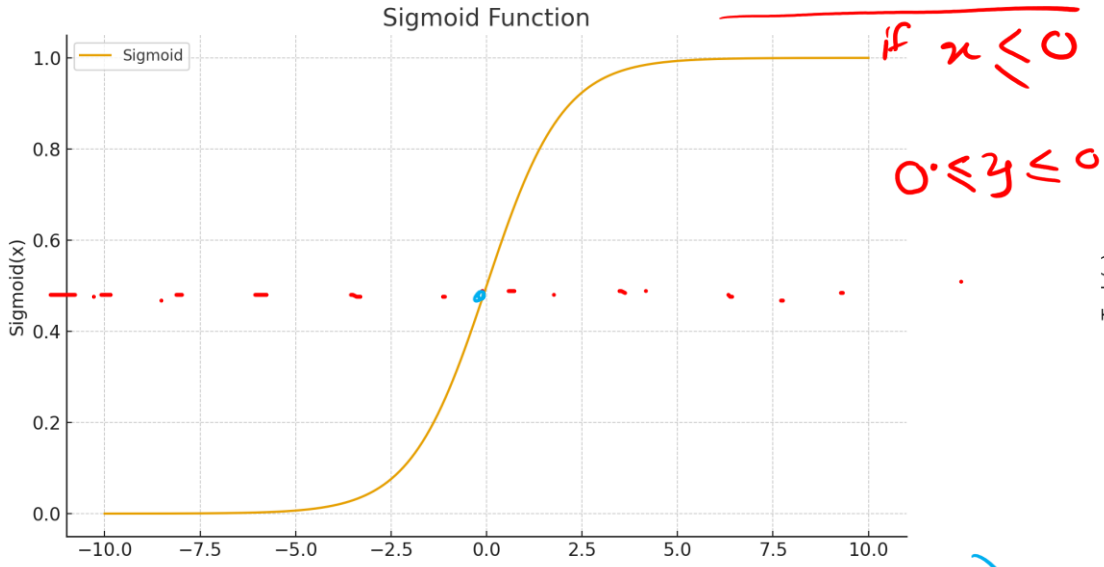
Common Activation Functions (Sigmoid, Tanh, ReLU, Leaky ReLU)

Softmax

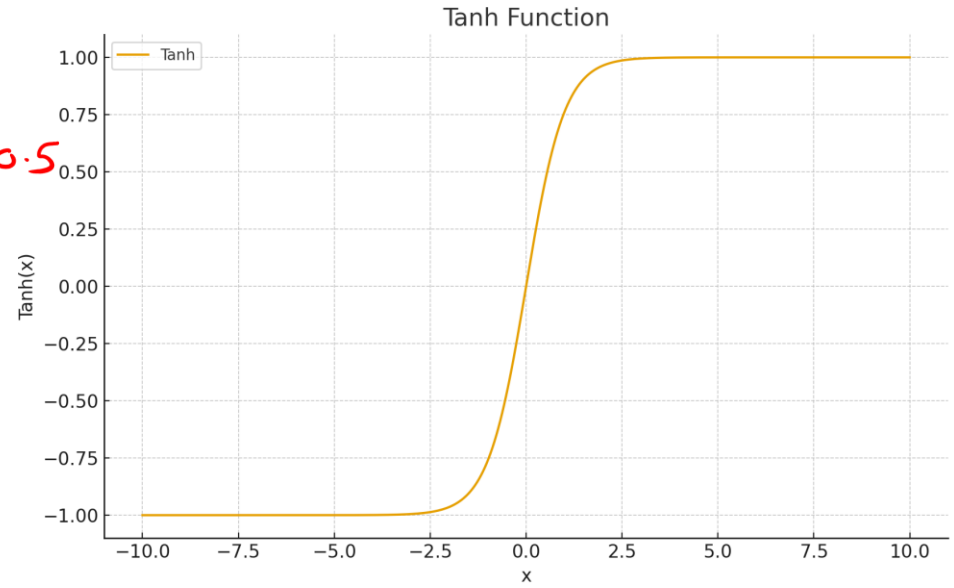
Signum



Sigmoid Function



Tanh Function



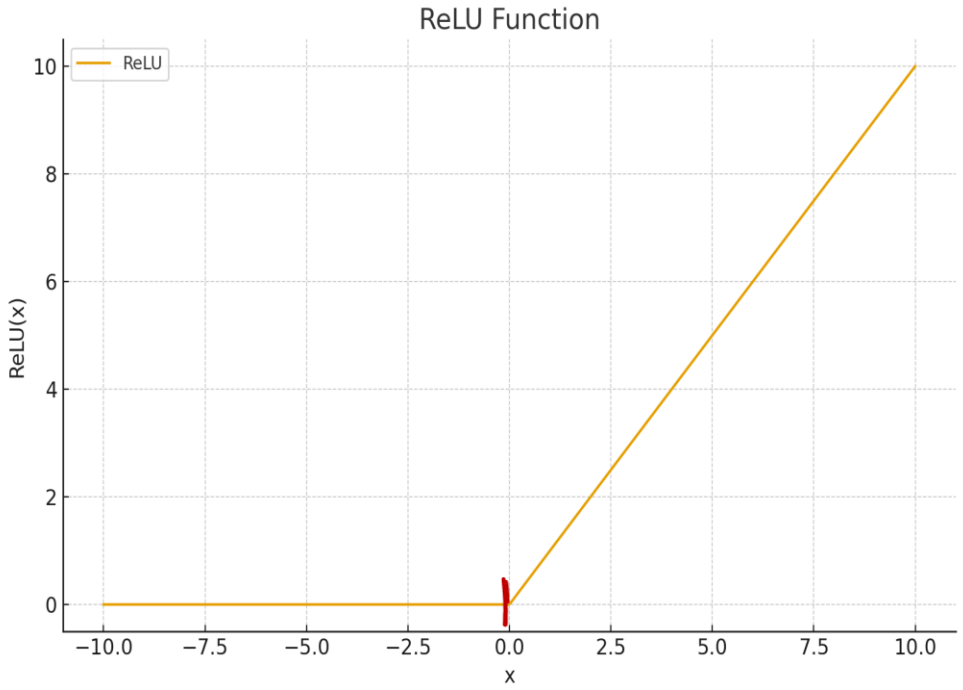
Common Activation Functions (Sigmoid, Tanh, ReLU, Leaky ReLU)

$$\sum x_n \cdot w_n > 0$$

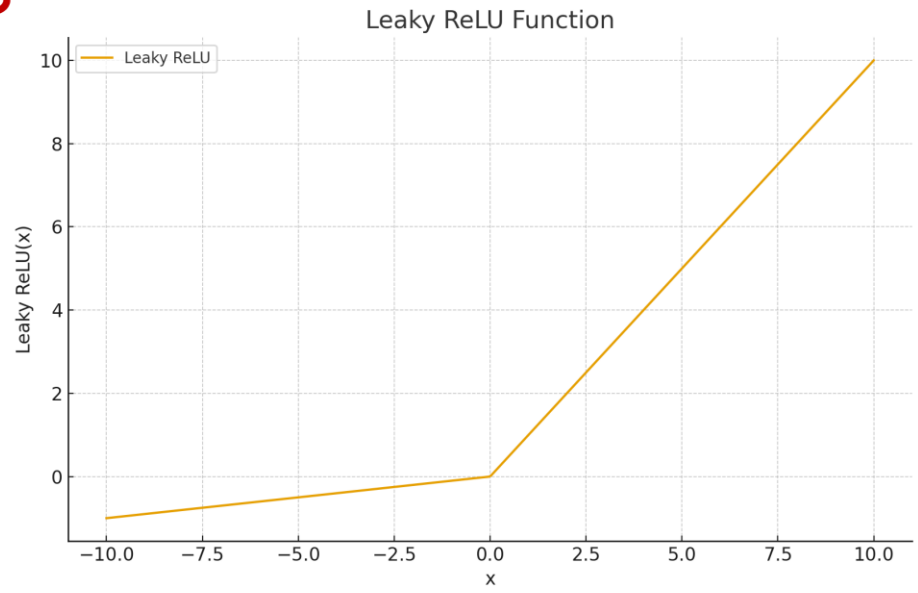
$$y \leq 0$$

$$y(x) = 0$$

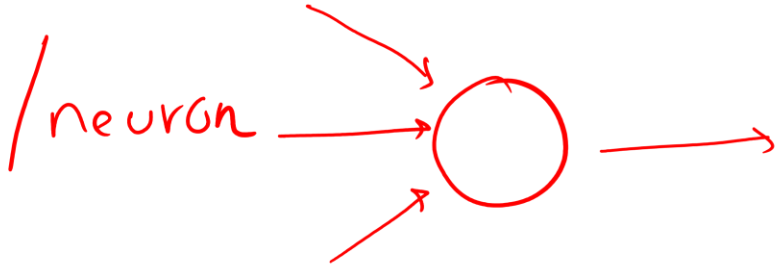
ReLU Function



Leaky ReLU Function



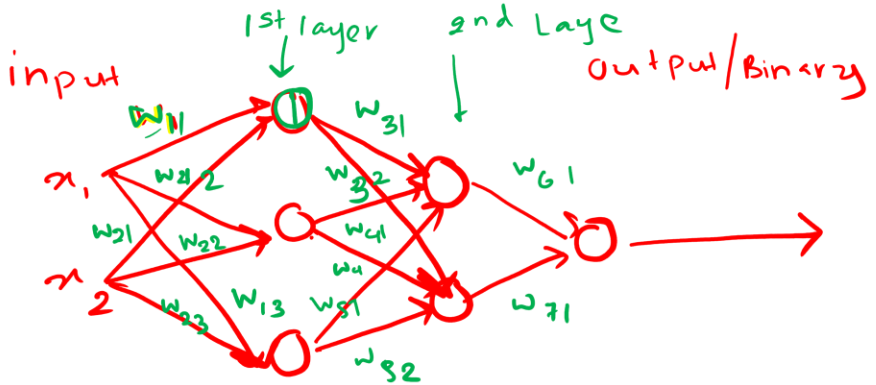
Limitations of Perceptron



- Cannot solve problems where data is **not linearly separable**.
  - Example: XOR (Exclusive OR) problem.
- Solution: Use **Multi-Layer Perceptrons (MLP)**.

draw back can solve only linear seperating data.

Multi Layer Neural Networks



- A multilayer neural network is an advanced model used in artificial intelligence and machine learning. Think of it as a collection of interconnected perceptrons.
- Unlike a perceptron, which has only one layer of neurons, a **multilayer neural network has multiple layers stacked on top of each other**.
- Each layer receives input from the previous layer and applies a mathematical operation called an activation function, such as the sigmoid function.
- Since all inputs are densely connected to all outputs, these layers are called **Dense Layers.** / Fully connected Layers

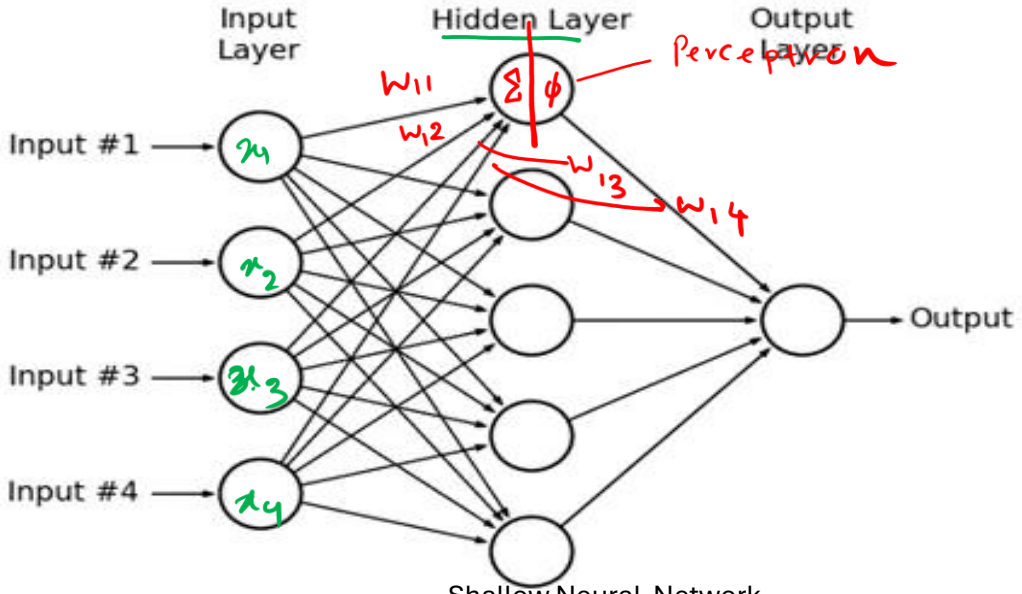
Types of Neural Networks

According to the no. of hidden Layers

(1)
- **Shallow neural networks** usually have only one hidden layer

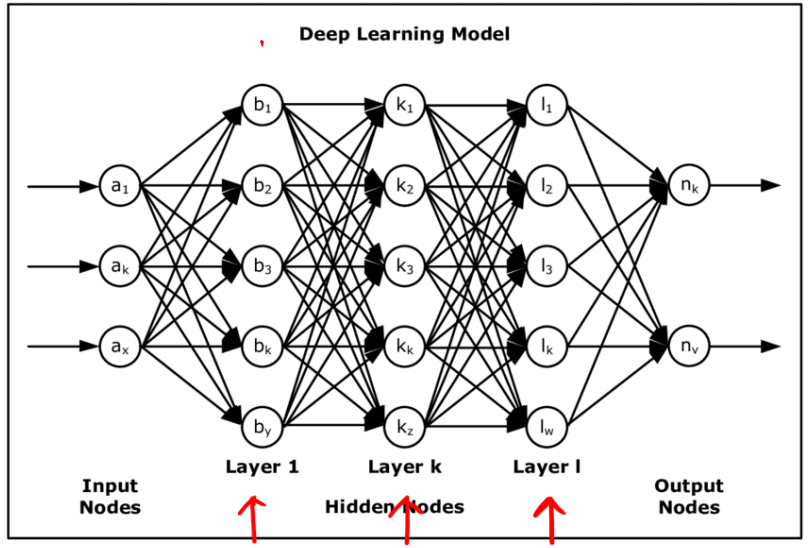
Shallow neural networks are fast and require less processing power than deep neural networks, but they cannot perform as many complex tasks as deep neural networks.

(2)
- **Deep neural networks** have multiple hidden layers



Shallow Neural Network

Source; https://medium.com/machine-learning-researcher/artificial-neural-network-ann-4481fa33d85a



Deep Neural Network

Source: Serrano, W. (2017). Smart internet search with random neural networks. *European Review*, *25*(2), 260-272.

Multi-Layer Neural Networks (MLP); Structure

Input Layer

- The first layer that takes in raw features from the dataset.

Example: In image recognition, each pixel value is an input.

Hidden Layers

- Intermediate layers that transform inputs into useful representations.
- Each hidden layer applies **weights + biases + activation functions** to the inputs.
- The deeper the layers, the more **abstract patterns** the network can learn.

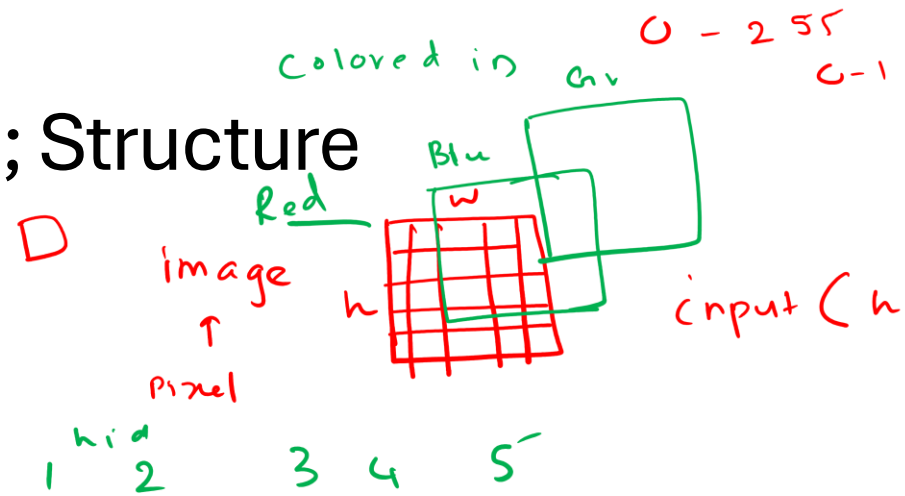
Example (for image recognition):

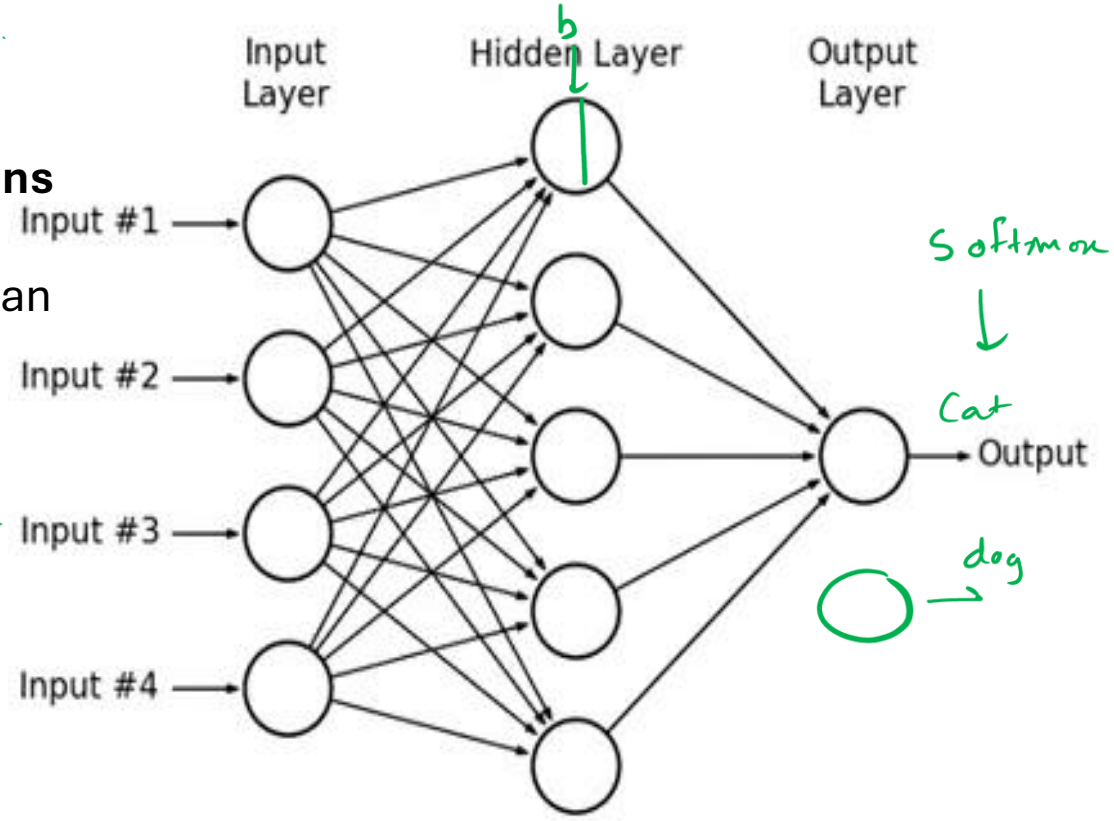
Hidden Layer 1 → detects edges (lines, corners). → low level feature
Hidden Layer 2 → detects shapes (circles, rectangles). → mid level feature
Hidden Layer 3 → detects objects (faces, cars, animals). → high level

Output Layer

- Produces the final decision/prediction. [0.5] cat /or not

Example: Cat vs. Dog classification → probabilities like [0.9, 0.1].





building a Simple neural network from Scratch

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 4 → 5 → 1 MLP
model = keras.Sequential([
    layers.Input(shape=(4,)),          # Input layer: 4 features
    layers.Dense(5, activation="relu"),# Hidden layer: 5 neurons (ReLU)
    layers.Dense(1, activation="sigmoid")  # Output layer: 1 neuron
])
```

Provide input list

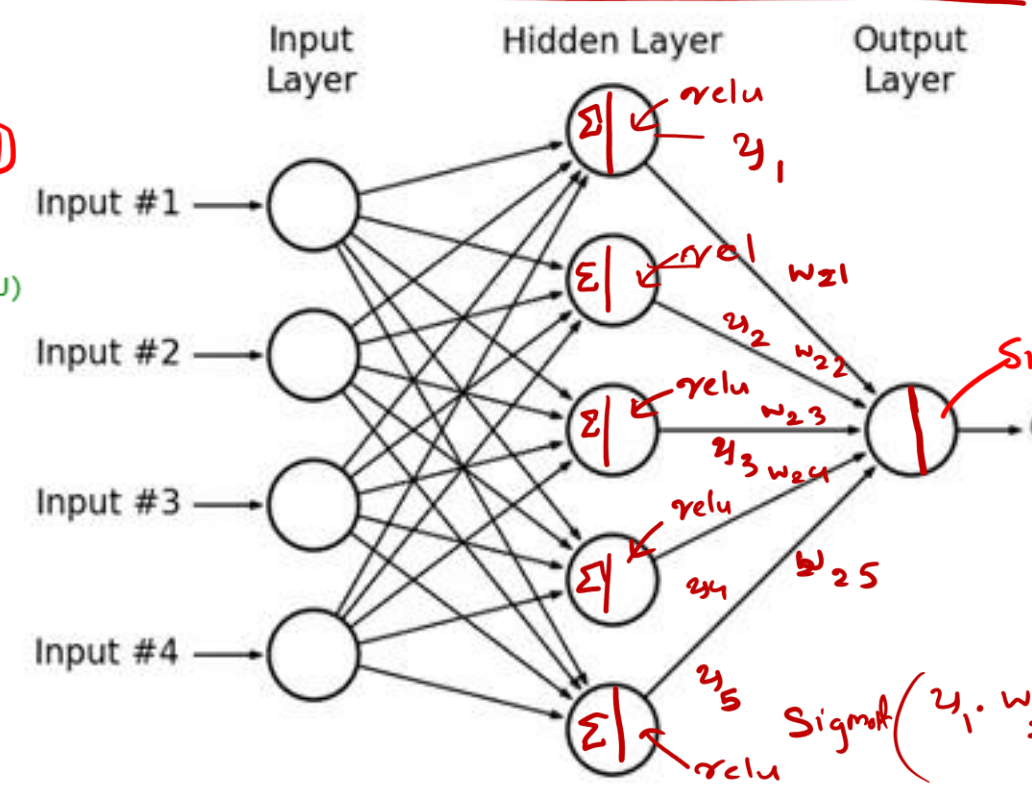
model.s Keras. Sequel

Softmax

Pytorch

```python
import torch
import torch.nn as nn

# Define the model
model = nn.Sequential(
    nn.Linear(4, 5),    # Input layer: 4 features → Hidden layer: 5 neurons
    nn.ReLU(),          # Hidden layer activation
    nn.Linear(5, 1),    # Output layer: 1 neuron
    nn.Sigmoid()        # Output activation (binary classification)
)
```



1st →
2nd =

Cat / not

Sigmoid → (0.71)

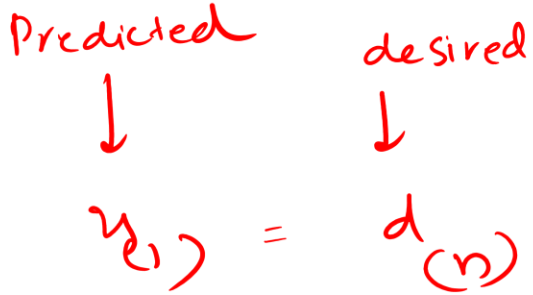
0.7 → 1

0.32 → 0

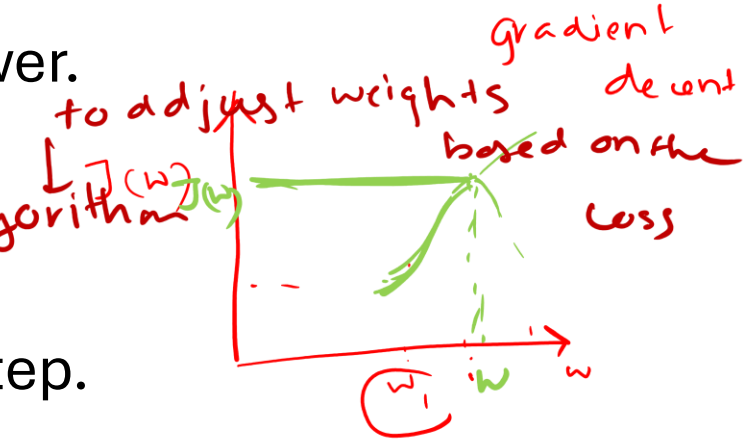
Sigmoid($y_1 \cdot w_{21} + y_2 \cdot w_{22} + y_3 \cdot w_{23} + y_4 w_{24} + y_5 w_{25}$)

Forward propagation (calculate Predicted output)

How Neural Networks Learn



- **Step 1: Forward Propagation**
  - Data flows from input → hidden layers → output.
    - Example: An image is processed layer by layer, from pixels to object recognition.
- **Step 2: Loss Function**
  - Measures error between prediction and actual answer.
    - Example: Predicted: "Cat" (0.7), Actual: "Dog". not a cat
- **Step 3: Backpropagation** / backpropagation algorithm
  - Error is sent backward to update weights.
  - Uses **Gradient Descent** to minimize error step by step.
- **Step 4: Training**
  - Repeat forward + backward many times until the network learns.



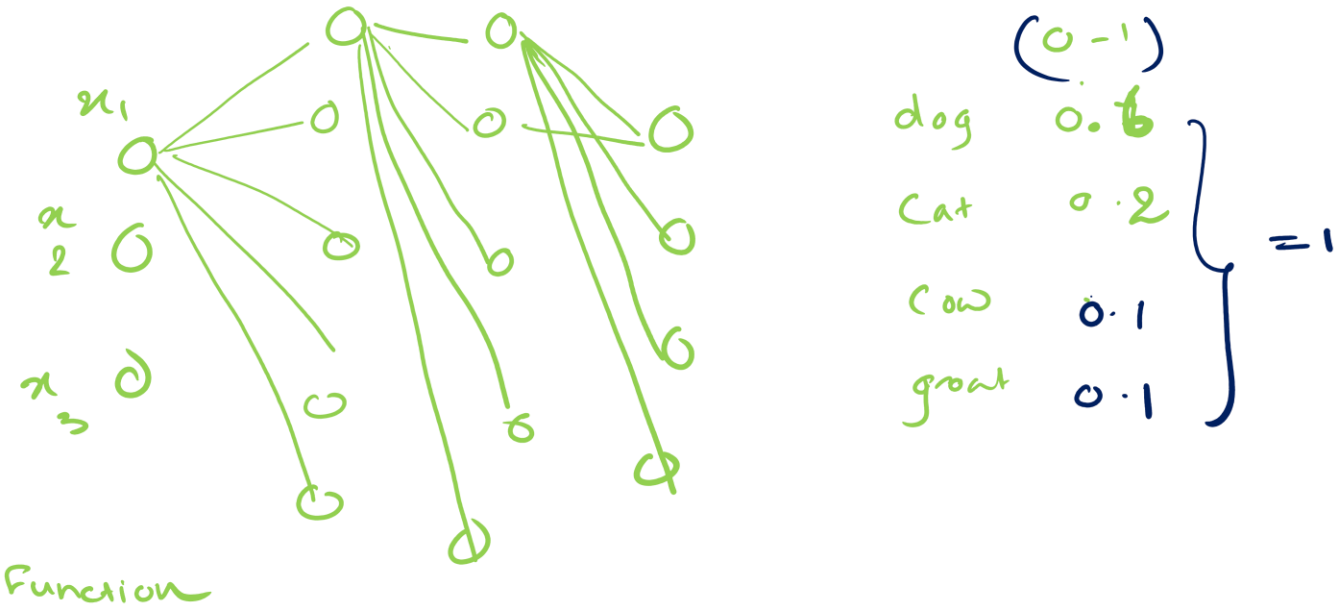
forward propagation (make a prediction)

Idea

Data flows layer→layer.

- Each neuron does a weighted sum, adds a bias, then applies an activation (e.g., ReLU/sigmoid).
- The outputs of one layer become the inputs to the next.

- Math (for layer *l*)
  - weights & bias: $W^{(l)}$, $b^{(l)}$
  - pre-activation: $z^{(l)}=W^{(l)}a^{(l-1)}+b^{(l)}$
  - activation: $a(l)=f(z^{(l)})$
  - output layer:
    - binary classification: $\hat{y}=\sigma(z)$
    - multi-class: $\hat{y}=\text{softmax}(z)$

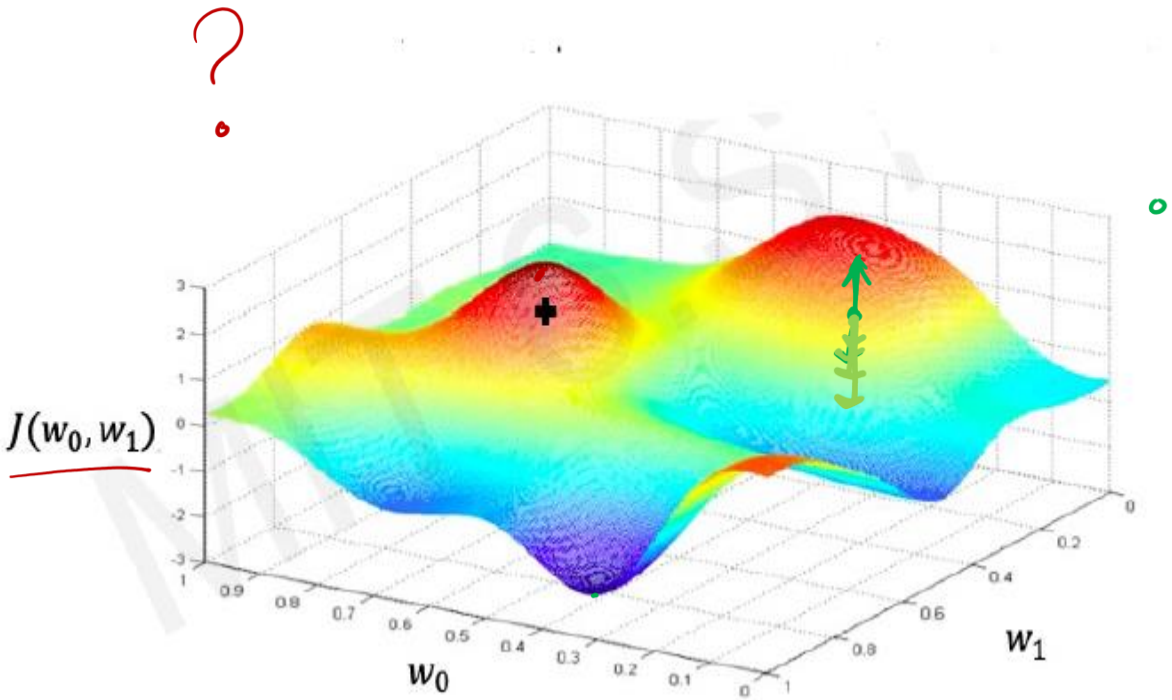


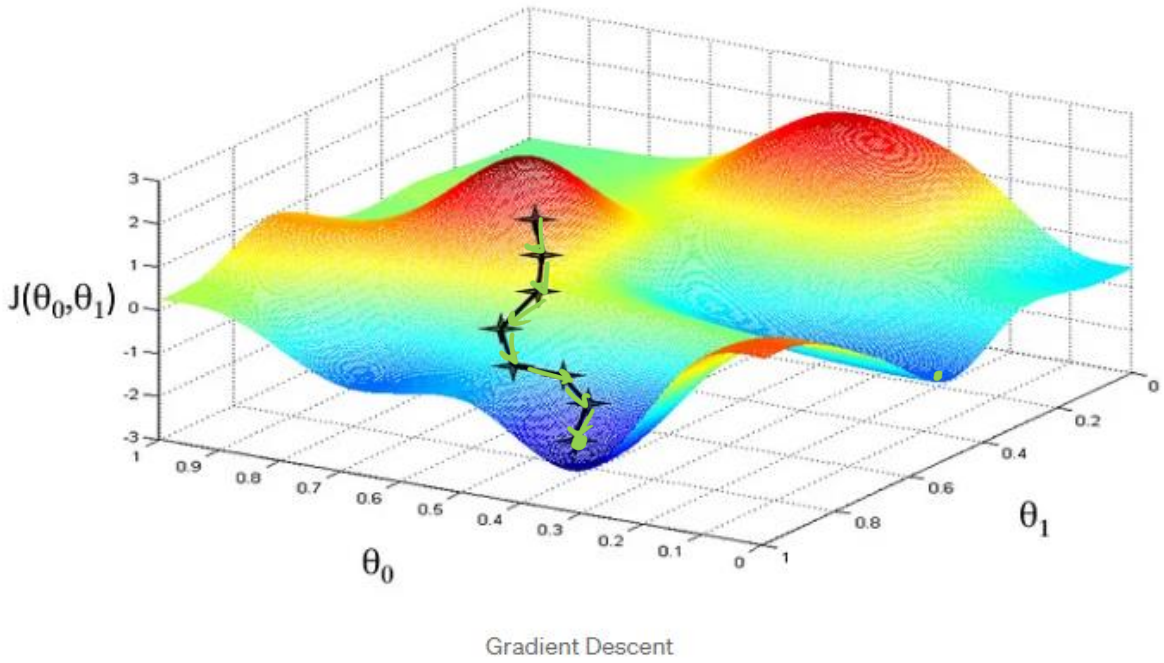
loss function (measure “how wrong”)

- **Idea**
  compare prediction y^ with true label y. The loss is a single number to minimize.

- **Common choices**
  - **binary cross-entropy (BCE)** for 2 classes:
    $L = -\left[y \log(\hat{y}) + (1-y) \log(1-\hat{y})\right]$
  - **categorical cross-entropy (CE)** for K classes:
    $L = -\sum_{k=1}^{K} y_k \log(\hat{y}_k)$ (with one-hot y)
  - **mse** (regression): $L = \frac{1}{n}\sum(\hat{y} - y)^2$

Gradient Descent

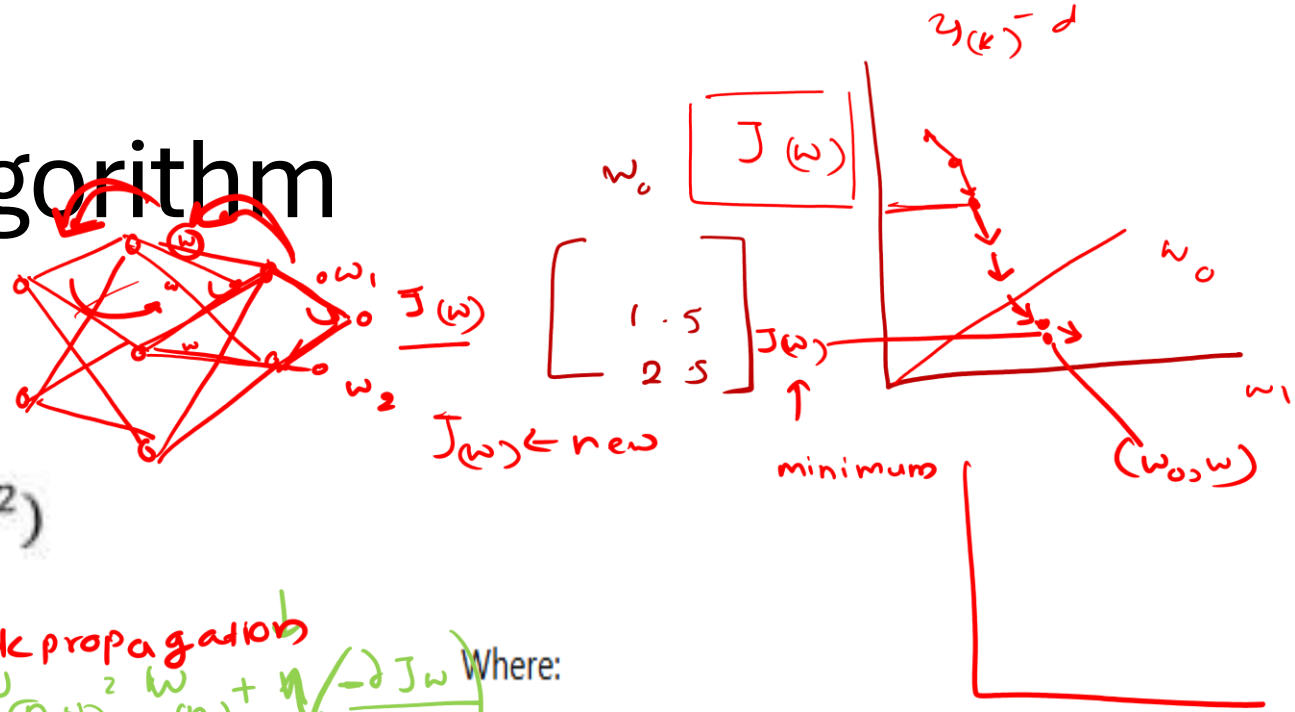




Gradient Descent

Update the parameters using the partial derivatives of the loss function with respect to the parameters (W and b).

Gradient Descent algorithm



Algorithm

1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

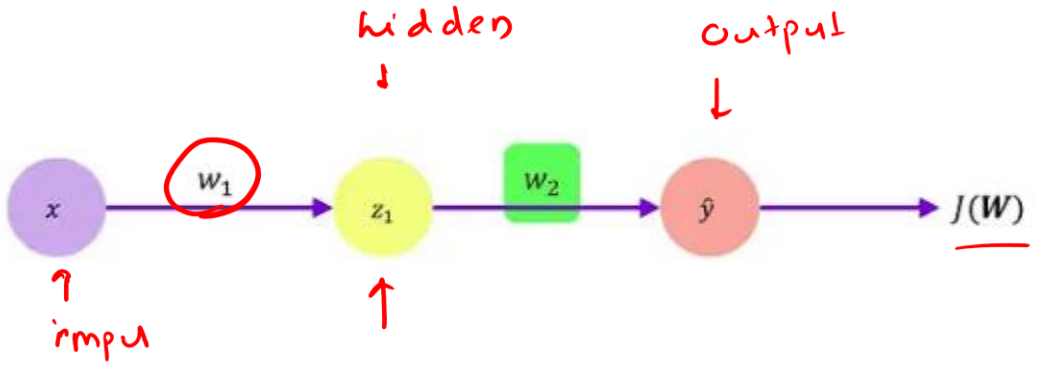
Where:

- W = current weight
- η = learning rate (step size)
- $\frac{\partial L}{\partial W}$ = gradient of the loss w.r.t the weight

Source: https://medium.com/analytics-vidhya/neural-networks-part-2-building-neural-networks-understanding-gradient-descent-145718e91270

backpropagation (compute gradients)

J(W)



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial w_1}$$

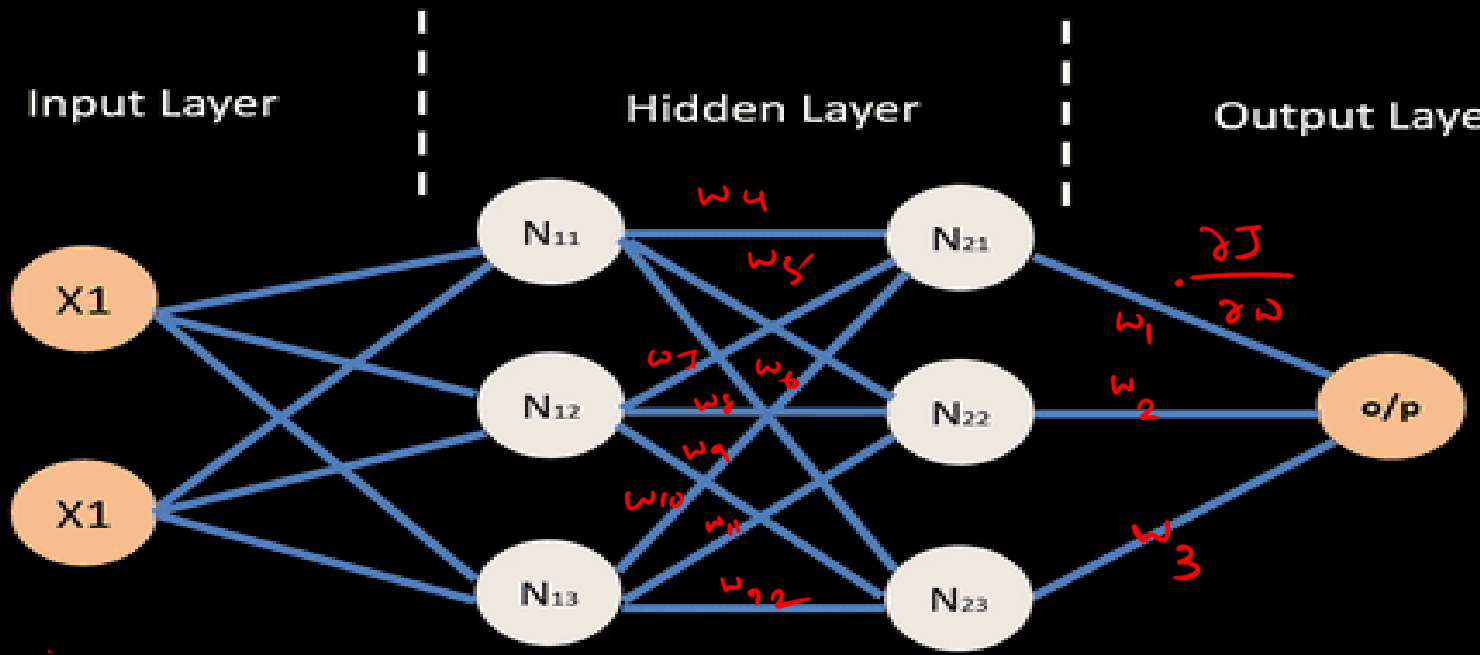


$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial w_1}$$

Backpropgate all these gradients from output to input

Source: https://medium.com/analytics-vidhya/neural-networks-part-3-understanding-back-propagation-learning-rates-3482a981a2f0

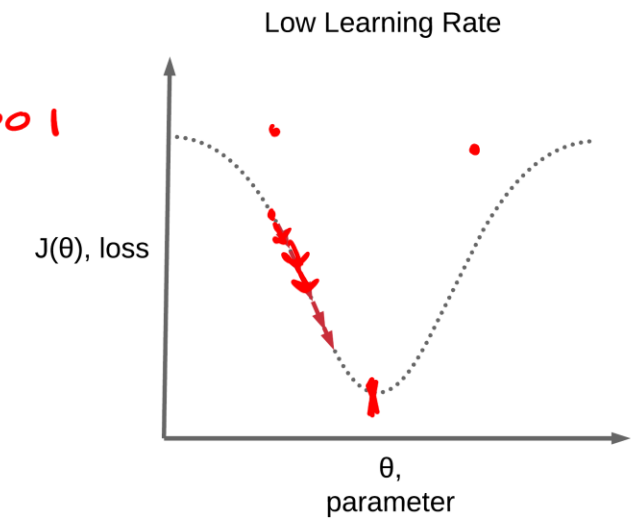
Neural Network – Backpropagation

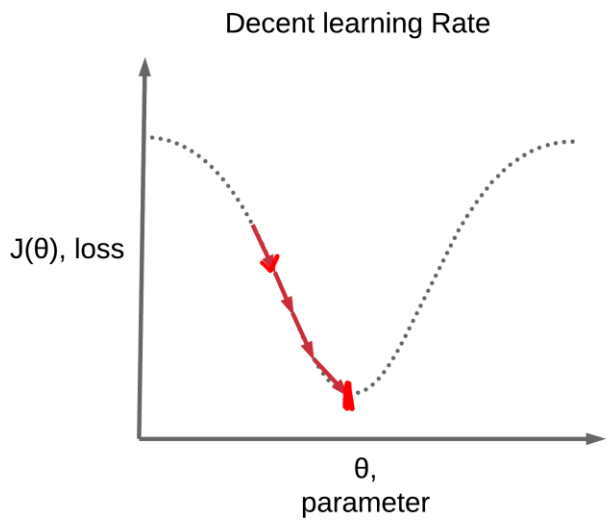


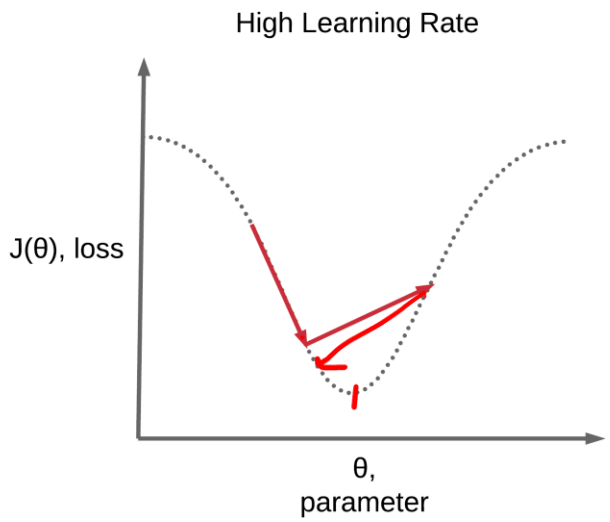
© machinelearningknowledge.ai

Learning Rate

- The learning rate is a hyperparameter that controls the speed at which a neural network learns.
- The learning rate is a measure of how much the weights of the neural network are updated each time the model is trained.







Techniques for Handling Learning Rate

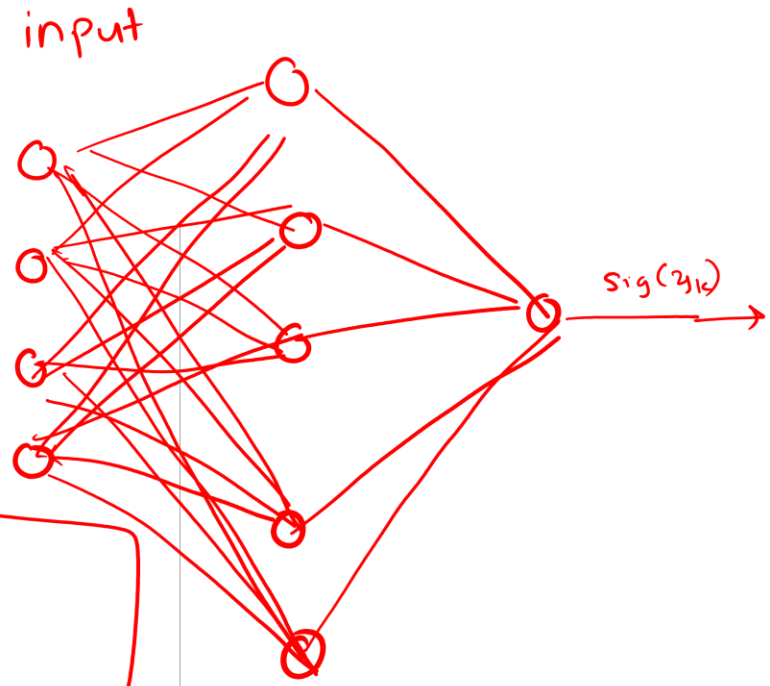
- **Learning Rate Finder** randomly

  Empirically search for a suitable learning rate by testing across a range and observing loss behavior.

- **Adaptive Learning Rate Methods**

  Optimizers like **Adam, RMSprop, AdaGrad** adjust the learning rate for each parameter automatically (that “adapts” to the landscape).

Can you draw architecture?




```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

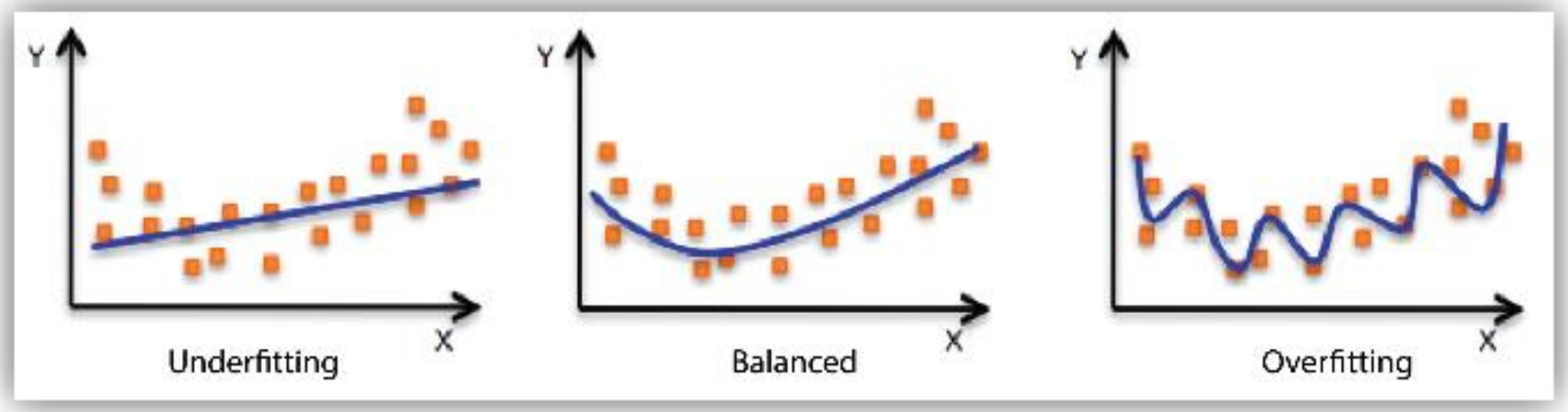
# 4 → 5 → 1 MLP
model = keras.Sequential([
    layers.Input(shape=(4,)),          # Input layer: 4 features
    layers.Dense(5, activation="relu"),# Hidden layer: 5 neurons (ReLU)
    layers.Dense(1, activation="sigmoid")  # Output layer: 1 neuron
])

model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.summary()
```

1
2
3
binary class
hyper parameter

Good Practices in Neural Networks

- Use Mini –Batches
  - More accurate estimation of gradient
  - Fast training
- The problem of overfitting



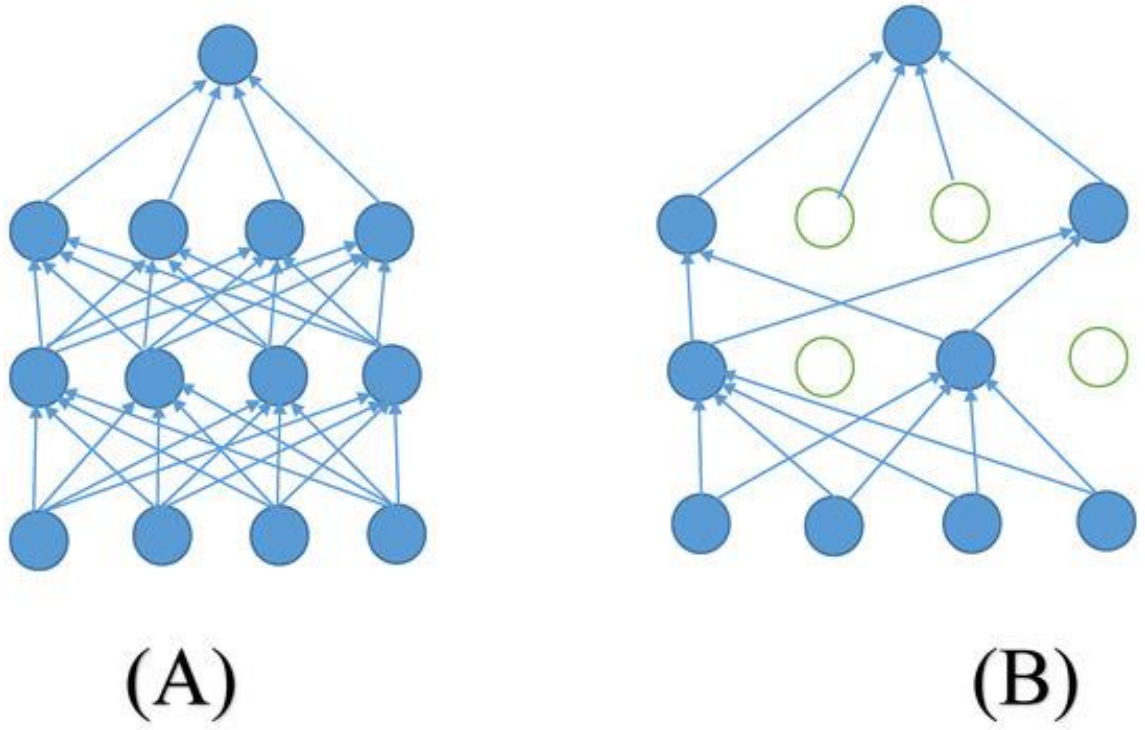
How to avoid over-fitting

- Adding dropouts

```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 4 → 8 → 8 → 1 with Dropout
model = keras.Sequential([
    layers.Input(shape=(4,)),          # Input layer: 4 features
    layers.Dense(4, activation="relu"),
    layers.Dropout(0.5),               # Drop 50% of units
    layers.Dense(4, activation="relu"),
    layers.Dropout(0.5),               # Drop 50% again
    layers.Dense(1, activation="sigmoid")  # Output layer: 1 neuron
])

model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.summary()
```



An illustration of a standard neural network (**A**) and after applying dropout (**B**). Source: Salehin, I., & Kang, D. K. (2023). A review on dropout regularization approaches for deep neural networks within the scholarly domain. *Electronics*, *12*(14), 3106.

How to avoid over-fitting

- Early stopping

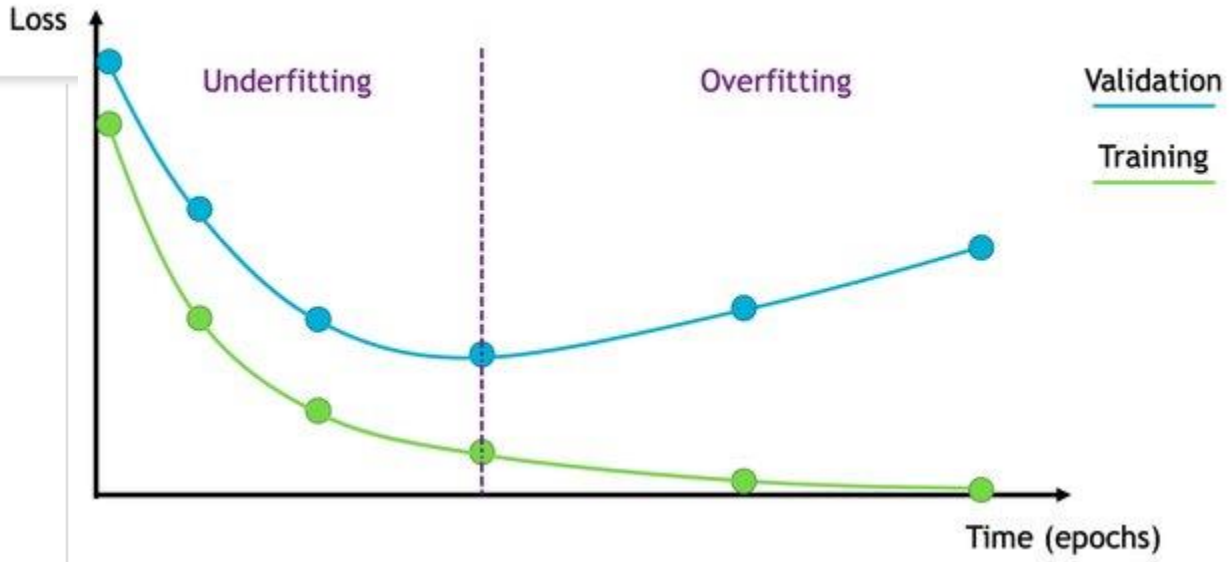
```python
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 4 → 4 → 4 → 1 with Dropout
model = keras.Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(4, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(4, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(1, activation="sigmoid")
])

model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

# --- Early Stopping ---
early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss",        # watch validation loss
    patience=5,                # stop if no improvement for 5 epochs
    mode="min",
    restore_best_weights=True  # use the best model seen during training
)

model.summary()
```

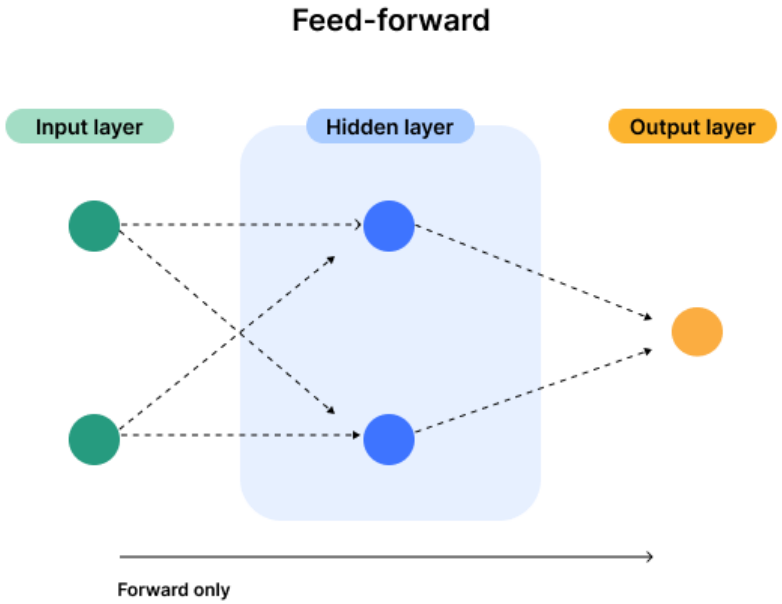


Sournce: Vakalopoulou, M., Christodoulidis, S., Burgos, N., Colliot, O., & Lepetit, V. (2023). Deep learning: basics and convolutional neural networks (CNNs) 2023. *Machine Learning for Brain Disorders [Internet]. New York, NY, Humana*.

Types of Neural Networks

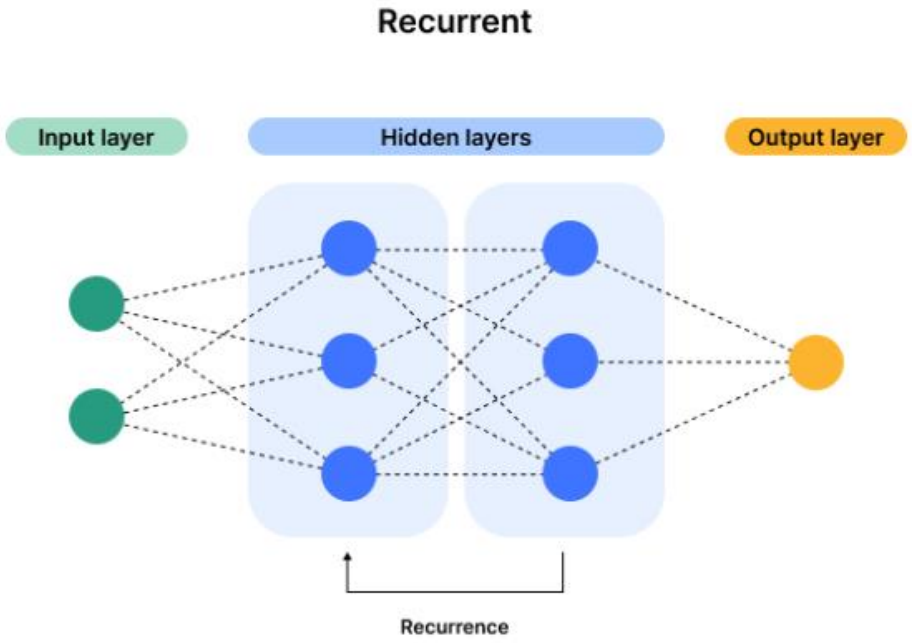
- **Feedforward Neural Network (FNN)**
  - **How it works:** Information moves in one direction—from input → hidden layers → output.
  - **Use cases:** Basic classification (e.g., predicting if an email is spam or not).
  - **Key point:** It is the simplest type of neural network, no loops.

- **Convolutional Neural Network (CNN)**
  - **How it works:** Uses filters (kernels) to scan through data, especially images, and detect patterns like edges, shapes, or textures.
  - **Use cases:** Image recognition (e.g., recognizing cats vs. dogs), video analysis, medical image analysis.
  - **Key point:** Best suited for image and spatial data.



Types of Neural Networks

- **Recurrent Neural Network (RNN)**
  - **How it works:** Has connections that loop back, meaning it remembers information from previous steps.
  - **Use cases:** Language modeling, text prediction, speech recognition, time-series forecasting.
  - **Key point:** Good for sequential data where the order matters.
- **Long Short-Term Memory (LSTM) Networks**
  - **How it works:** A special type of RNN that avoids the problem of "forgetting" long sequences by using memory cells and gates.
  - **Use cases:** Machine translation, chatbot responses, predicting stock prices.
  - **Key point:** Designed to remember information for longer sequences.



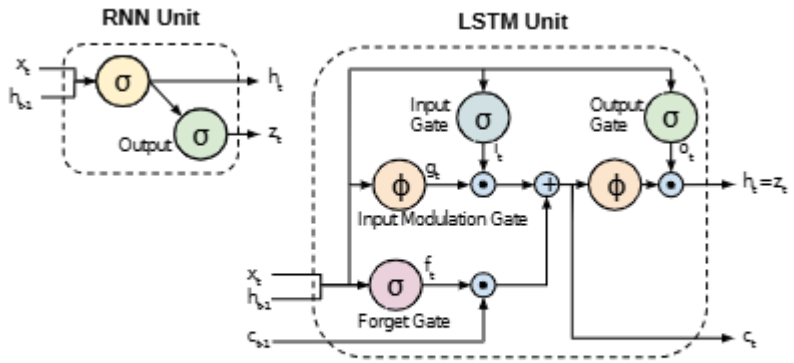


Figure 1: RNN simple cell versus LSTM cell [4]

Types of Neural Networks

Generative Adversarial Networks (GANs)

- **How it works:** Made up of two networks—**Generator** (creates fake data) and **Discriminator** (checks if data is real or fake). They compete with each other to improve.
- **Use cases:** Creating realistic images, deepfakes, art, data augmentation.
- **Key point:** Good at generating new, realistic data.

Self-Organizing Maps (SOMs)

- **How it works:** An unsupervised network that maps high-dimensional data into a lower dimension (usually 2D) for visualization.
- **Use cases:** Data clustering, feature mapping, pattern recognition.
- **Key point:** Helps visualize complex data.

Modular Neural Networks (MNNs)

- **How it works:** Splits a big problem into smaller parts, each solved by a separate network. Their results are combined at the end.
- **Use cases:** Large, complex systems such as robotics or medical diagnosis.
- **Key point:** Breaks down complex problems into manageable modules.